

Lovable Prompt: Build Standalone Fullscreen Media Player (Phase 1)

What to Build

Build a brand new, standalone fullscreen media player system at the route `/mediaplayer`. This is a completely independent system — do NOT import from, reference, or depend on any existing media/video components in the codebase. Everything must be built from scratch inside a new `src/components/media-system/` directory.

This is Phase 1 — the foundation. We need core playback working perfectly before adding polish.

Route Setup

Create a new route at `/mediaplayer` that renders the `FullscreenMediaViewer` component. This is a test/standalone page — it should be fullscreen (100dvh), dark background (#000), no header, no bottom nav. Just the media player.

For now, use hardcoded test data with real Cloudflare Stream HLS URLs from our existing posts. Query the `posts` table in Supabase to get 10-15 recent posts that have video content (where `media_urls` contains `.m3u8` URLs). Use these real URLs as the test feed data. If a post has multiple media items, include them all so we can test the carousel later. For posts without video, use their image URLs.

Core Architecture to Build

1. Video Pool Manager (`VideoPoolProvider.tsx` + `hooks/useVideoPool.ts`)

This is the most critical piece. Instead of each feed item creating its own `<video>` element (expensive on mobile), we maintain a pool of **3 reusable `<video>` elements** that get dynamically assigned to visible feed positions.

On mount, pre-create 3 `<video>` elements with these attributes:

- `playsInline: true`
- `webkit-playsinline: true` (legacy iOS)
- `preload: "auto"`
- `muted: true` (required for autoplay)
- `loop: false`
- `crossOrigin: "anonymous"`

These elements live in a hidden container and get **moved** (not recreated) to whichever feed item needs them. Use a React Context (`VideoPoolProvider`) to make the pool accessible to all child components.

Assignment logic: When the viewport observer detects a new active item, the Pool Manager:

1. Checks if the media URL is already assigned to a pool element (cache hit → instant)
2. If not, finds the pool element furthest from the viewport (LRU recycling)
3. Pauses the recycled element, stores its playback position in a session cache Map
4. Assigns the new HLS source, moves the element to the requesting container, plays

Session Cache: A `Map<string, { currentTime: number, wasMuted: boolean, duration: number }>` that persists for the page session. When a video element is reassigned to a URL that's in the cache, restore the playback position. This makes back-navigation instant.

2. Feed Container (`FeedContainer.tsx`)

A vertical scroll container that snaps to each feed item:

- Full viewport height items (100dvh per item)
- Use CSS `scroll-snap-type: y mandatory` with `scroll-snap-align: start` on each item
- Dark background (#000)
- `overflow-y: scroll` with `-webkit-overflow-scrolling: touch`
- Hide scrollbar with CSS

Each feed item takes up exactly one full screen. The container handles the vertical feed scroll.

3. Viewport Observer (`hooks/useViewportObserver.ts`)

Use IntersectionObserver to detect which feed item is currently visible:

- Threshold: `[0, 0.5, 1.0]`
- When an item crosses 50% visibility, it becomes the “active” item
- The active item’s video plays, all others pause
- Include a 150ms debounce to prevent rapid switching during fast scrolling
- Track scroll direction by comparing which items are gaining/losing visibility

When active item changes:

1. Pause the previous active video
2. Tell the Pool Manager to assign a video element to the new active item
3. Play the new active video

4. HLS Integration (`utils/hlsManager.ts`)

Create an HLS manager utility:

- Detect if the browser supports native HLS
(`video.canPlayType('application/vnd.apple.mpegurl')`)
- For Safari/iOS: set `video.src` directly to the `.m3u8` URL
- For all other browsers: use `hls.js` library

hls.js config:

```
{
  enableWorker: true,
  lowLatencyMode: false,
  startLevel: -1,           // auto-select quality
  capLevelToPlayerSize: true,
  maxBufferLength: 15,
  maxMaxBufferLength: 30,
  startFragPrefetch: true,
  progressive: true,
  abrBandwidthFactor: 0.95,
  abrBandwidthUpFactor: 0.7,
  fragLoadingMaxRetry: 4,
}
```

Install hls.js if not already installed: `npm install hls.js`

The manager should expose: `attachMedia(videoElement, hlsUrl)` ,
`detachMedia(videoElement)` , and `destroy()` .

5. Individual Video Player (`VideoPlayer.tsx`)

A component that:

- Receives a media URL and a container ref
- Requests a video element from the Pool Manager
- Positions the video element to fill its container (`object-fit: cover` , absolute positioned, full width/height)
- Handles the `canplay` event to know when playback is ready
- Shows a loading skeleton until the video is ready to play
- On tap: toggle play/pause with a brief play/pause icon overlay (show for 800ms then fade)

6. Loading Skeleton (`LoadingSkeleton.tsx`)

A full-screen skeleton that shows while video is loading:

- Dark background matching the app (`#1a1a1a` to `#0d0d0d` gradient)
- Shimmer animation: a subtle light sweep moving left to right

```
@keyframes shimmer {
  0% { background-position: -200% 0; }
  100% { background-position: 200% 0; }
}

background: linear-gradient(
  90deg,
  rgba(255,255,255,0.0) 25%,
  rgba(255,255,255,0.08) 50%,
  rgba(255,255,255,0.0) 75%
);
background-size: 200% 100%;
animation: shimmer 1.5s infinite ease-in-out;
```

The skeleton fades out (200ms) when the video fires `canplay` , and the video simultaneously fades in.

7. Basic Scrubber (`scrubber.tsx`)

A thin progress bar at the bottom of each video:

- Default: 2px height, white at 60% opacity, showing played progress
- Shows buffered range in lighter white (30% opacity)
- For now, just a visual indicator (seeking interaction comes in Phase 3)
- Position it at the very bottom of the feed item, above any safe area

8. Mute/Unmute Button

A single button in the top-right corner:

- Speaker icon (unmuted) / speaker with slash (muted)
- 44x44px touch target
- Positioned 16px from right, 16px below safe-area-inset-top
- Global state: toggling mute/unmute applies to all videos in the session
- Default state: muted (required for autoplay)
- Use Zustand or simple React context for the global mute state

State Management

Create a Zustand store at `src/components/media-system/store/mediaStore.ts` :

```
interface MediaStore {
  activeIndex: number;
  isMuted: boolean;
  setActiveIndex: (index: number) => void;
  toggleMute: () => void;
}
```

Keep it minimal for Phase 1. We'll expand it later.

File Structure

Create all files under `src/components/media-system/` :

```
src/components/media-system/
├─ FullscreenMediaViewer.tsx
├─ VideoPoolProvider.tsx
├─ FeedContainer.tsx
├─ FeedItem.tsx
├─ VideoPlayer.tsx
├─ LoadingSkeleton.tsx
├─ Scrubber.tsx
├─ MuteButton.tsx
├─ hooks/
│   └─ useVideoPool.ts
│   └─ useViewportObserver.ts
│   └─ useSessionCache.ts
├─ store/
│   └─ mediaStore.ts
├─ utils/
│   └─ hlsManager.ts
└─ types/
    └─ media.ts
```

Types (`types/media.ts`)

```
export interface MediaItem {
  id: string;
  type: 'video' | 'image';
  hlsUrl?: string;
  mp4Url?: string;
  imageUrl?: string;
  thumbnailUrl?: string;
  width: number;
  height: number;
  duration?: number;
}
```

```
export interface FeedPost {
  id: string;
  userId: string;
  username: string;
  avatarUrl: string;
  caption: string;
  mediaItems: MediaItem[];
  likeCount: number;
  commentCount: number;
}
```

```
    isLiked: boolean;  
  }
```

Key Requirements

1. **Videos MUST autoplay when they become the active item** — muted autoplay works on all browsers
 2. **Videos MUST pause when they leave the viewport** — only one video playing at a time
 3. **The pool must REUSE elements** — never create more than 3 `<video>` elements total
 4. **Session cache must work** — scrolling back to a previous video should resume from where it left off
 5. **Skeleton shimmer must show** until the video is actually ready to play
 6. **Smooth skeleton→video transition** — crossfade, no flash, no layout shift
 7. **Mute state is global** — unmuting on one video unmutes all subsequent videos
 8. **The route `/mediaplayer` must be accessible** — add it to the router
-

What NOT to Build Yet (Phases 2-5)

- No preloading pipeline (Phase 2)
- No network-aware quality adaptation (Phase 2)
- No scrubber seeking/interaction (Phase 3)
- No spring physics animations (Phase 3)
- No horizontal carousel (Phase 4)
- No social overlay (like, comment, share) (Phase 4)
- No user info bar (Phase 4)
- No safe area handling (Phase 5)

Just get the core loop perfect: scroll → detect active → assign pool element → play HLS → show video → scroll to next → recycle element → repeat.

Before You Start

Please first audit how the current `MediaPreviewViewer` component works (the existing fullscreen media viewer that opens when tapping media), as it has some good patterns we refined over time. Look at how it handles HLS initialization and video element management. Take the good patterns but build everything as new, independent components — do not import or extend the existing viewer.